

Introduction

This project builds an end-to-end data pipeline for a **customer churn prediction** use case, following the medallion architecture (Bronze → Silver → Gold) on top of Hadoop/HDFS. Raw data from multiple heterogeneous sources is ingested, cleaned, conformed into a dimensional model, and ultimately exposed as an ML-ready feature set for churn modeling.

Scope

The scope covers the full pipeline from raw data ingestion up to a query-ready, dimensionally modeled Gold layer:

- Ingesting data from three independent source systems (relational DB, CSV, JSON) into a raw/Bronze HDFS layer using Apache NiFi.
- Registering the Bronze layer as external Hive tables for schema-on-read access and validation.
- Cleaning, standardizing, and transforming the raw data into a Silver layer using PySpark.
- Modeling a Galaxy Schema (conformed dimensions + multiple fact tables) in the Gold layer, including a Hybrid SCD Type 1/2 customer dimension.
- Producing verification queries to confirm data integrity at each stage.

Out of scope: the actual training/evaluation of the churn prediction model itself — the pipeline's output (`FACT_MONTHLY_USAGE`, with its `is_churned` target label) is the hand-off point to the ML workflow.

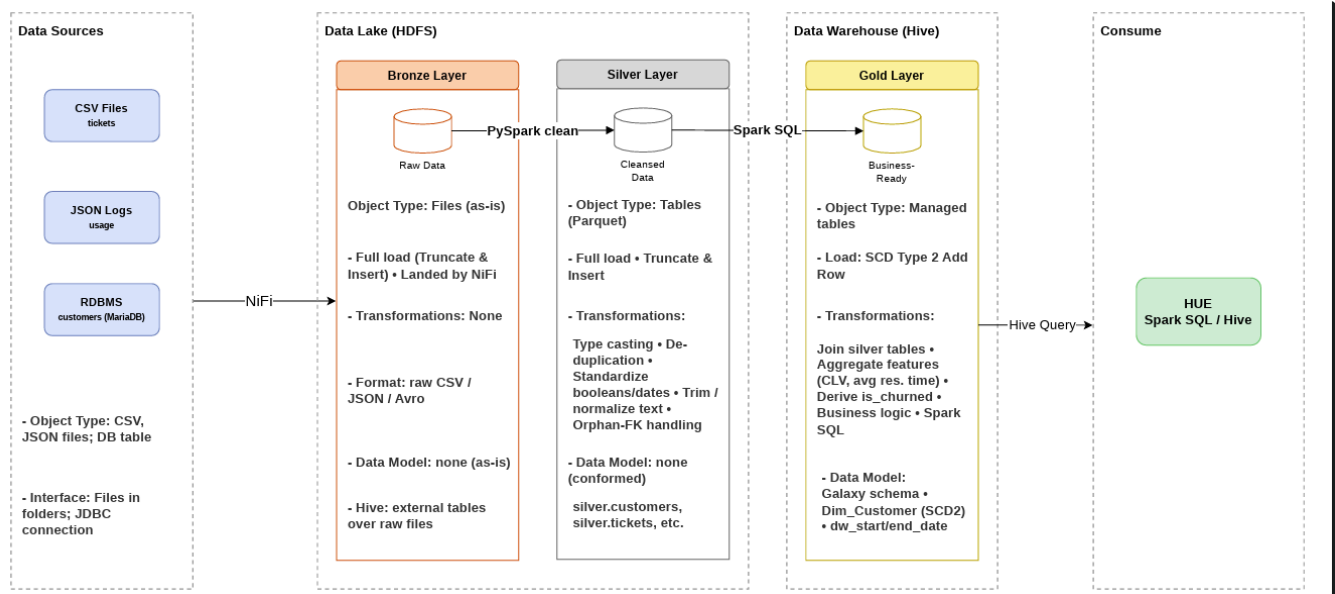
Problem Statement

The organization needs to predict which customers are likely to churn, but the relevant signals are scattered across disconnected systems: core customer/account data lives in a relational database, customer service friction is logged separately as support tickets, and product usage behavior is captured in usage logs. These sources differ in format (relational, CSV, JSON), update frequency, and grain (snapshot vs. transactional vs. periodic), making them unusable for modeling in their raw form. A unified, historically-accurate, analysis-ready dataset is needed — one that correctly tracks customer state over time (avoiding data leakage) and combines behavioral, demographic, and support signals into a single feature matrix.

Data Source

Source	Format	System	Bronze Table
Customer master data	Relational (MariaDB)	<code>source_rdbms.customers</code>	<code>bronze_customers</code> (Avro)
Customer support tickets	CSV	Flat file	<code>bronze_tickets</code>
Product usage logs	JSON	Flat file	<code>bronze_usage</code>

Pipeline

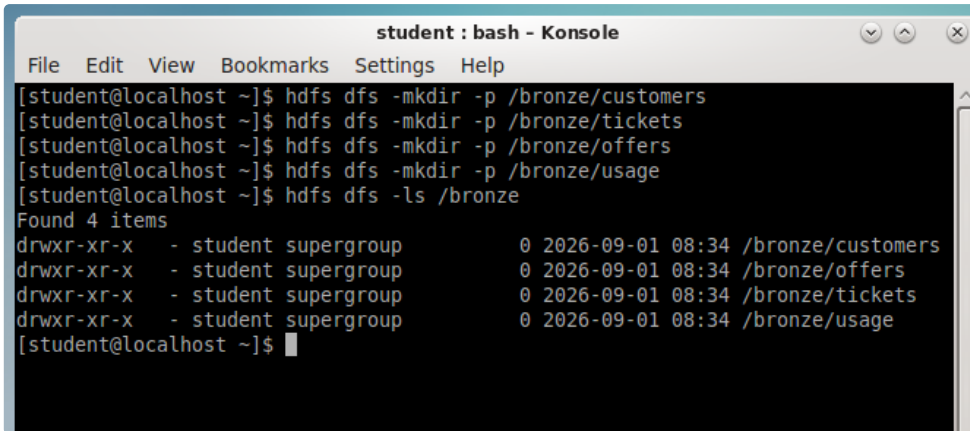


Bronze Layer

First create HDFS directories:

`bronze/offers` isn't needed anymore since we won't load that data source!!

```
hdfs dfs -mkdir -p /bronze/customers
hdfs dfs -mkdir -p /bronze/tickets
hdfs dfs -mkdir -p /bronze/offers
hdfs dfs -mkdir -p /bronze/usage
hdfs dfs -ls /bronze
```



```
student : bash - Konsole
File Edit View Bookmarks Settings Help
[student@localhost ~]$ hdfs dfs -mkdir -p /bronze/customers
[student@localhost ~]$ hdfs dfs -mkdir -p /bronze/tickets
[student@localhost ~]$ hdfs dfs -mkdir -p /bronze/offers
[student@localhost ~]$ hdfs dfs -mkdir -p /bronze/usage
[student@localhost ~]$ hdfs dfs -ls /bronze
Found 4 items
drwxr-xr-x  - student supergroup      0 2026-09-01 08:34 /bronze/customers
drwxr-xr-x  - student supergroup      0 2026-09-01 08:34 /bronze/offers
drwxr-xr-x  - student supergroup      0 2026-09-01 08:34 /bronze/tickets
drwxr-xr-x  - student supergroup      0 2026-09-01 08:34 /bronze/usage
[student@localhost ~]$
```

Copy all data sources files into the Windows/VM shared folder.

MySQL DB

Login to MariaDB:

```
mysql -u root -p
```

Create DB and exec the SQL file:

```
CREATE DATABASE source_rdbms;
USE source_rdbms;

##FILE LOCATION!!!
SOURCE /media/sf_Extra_files/Project/Datasets/churn_source_mysql;
```

```
SHOW TABLES;
SELECT * FROM customers LIMIT 5;
```

NiFi

CSV Ingestion Flow (Tickets Data)

We start by pulling the raw support ticket CSVs off the local filesystem and landing them in HDFS untouched.

GetFile processor

Add a `GetFile` processor and point it at the source directory:

- **Input Directory:** `/home/student/Project1/Datasets/`
- **File Filter:** `customer_support_tickets_light_dirty\.csv`
- **Keep Source File:** `true` — we don't want NiFi deleting the originals after pickup, since we may need to re-run ingestion during testing.

PutHDFS processor

Add a `PutHDFS` processor to write the picked-up file into the bronze layer:

- **Hadoop Configuration Resources:** `/home/hadoop/hadoop/etc/hadoop/core-site.xml,/home/hadoop/hadoop/etc/hadoop/hdfs-site.xml`
- **Directory:** `/bronze/tickets`

Auto-terminate both `success` and `failure` relationships on `PutHDFS` so the flow doesn't queue up dead connections. Then wire `GetFile → PutHDFS` on the `success` relationship, select both processors, and start them.

JSON Ingestion Flow (Usage Data)

Rather than build this from scratch, it's faster to clone the working CSV flow and repoint it — the two flows are structurally identical, just different filters and directories.

1. Stop the running `GetFile / PutHDFS` pair, copy them, and paste a duplicate onto the canvas.
2. On the cloned `GetFile`, update **File Filter** to `usage_logs_light_dirty\.json`.
3. On the cloned `PutHDFS`, update **Directory** to `/bronze/usage`.
4. Start both processors.

Verifying ingestion

Once both flows have run, confirm the files actually landed before moving on:

bash

```
hdfs dfs -ls /bronze/tickets
hdfs dfs -ls /bronze/usage
```

Fix HDFS permissions to write on student user:

```
hdfs dfs -chmod -R 777 /bronze
```

Start the NiFi processes, then boom — it loads into our HDFS datalake bronze layer (validation):

```
[student@localhost ~]$ hdfs dfs -ls /bronze/usage
Found 1 items
-rwxrwxrwx  1 root supergroup    6568203 2026-09-02 08:20 /bronze/usage/usage_logs_light_dirty.json
[student@localhost ~]$ hdfs dfs -ls /bronze/tickets
Found 1 items
-rwxrwxrwx  1 root supergroup    4777250 2026-09-02 08:20 /bronze/tickets/customer_support_tickets_light_dirty.csv
[student@localhost ~]$
```

MariaDB → HDFS Flow (DB Streaming)

First we need to download some jar files.

```
sudo wget -O /usr/local/nifi/nifi-1.14.0/lib/mariadb-java-client-3.1.4.jar  
https://repo1.maven.org/maven2/org/mariadb/jdbc/mariadb-java-client/3.1.4/mariadb-java-client-3.1.4.jar
```

```
ls -l /usr/local/nifi/nifi-1.14.0/lib/ | grep mariadb
```

Then Inside the `Customer_Churn_Pipeline` canvas, we build a second flow to pull the customer records straight from MariaDB rather than a flat file — this is the "streaming" source in our bronze layer.

QueryDatabaseTable processor

Drag a `QueryDatabaseTable` processor onto the canvas and configure its properties.

First we need a connection pool, since `QueryDatabaseTable` doesn't talk to the DB directly — it delegates to a **Database Connection Pooling Service**. From the dropdown, create a new `DBCConnectionPool`, then open its config:

- **Database Connection URL:** `jdbc:mariadb://localhost:3306/source_rdbms`
- **Database Driver Class Name:** `org.mariadb.jdbc.Driver`
- **Database Driver Location(s):** `/opt/nifi/nifi-current/lib/mariadb-java-client.jar` (wherever the jar actually landed)
- **Database User:** `root`
- **Password:** the MariaDB root password

Apply, then enable the pool via the lightning bolt icon — the processor won't run against a disabled controller service.

Back on the processor itself:

- **Table Name:** `customers`
- **Fetch Size:** `0` (no limit — pull everything)

Click **Apply**.

PutHDFS processor

Add a second `PutHDFS` processor and connect the `success` relationship from `QueryDatabaseTable` into it. Configure:

- **Hadoop Configuration Resources:** `/home/hadoop/hadoop/etc/hadoop/core-site.xml,/home/hadoop/hadoop/etc/hadoop/hdfs-site.xml`
- **Directory:** `/bronze/customers`

Under **Settings**, auto-terminate `success` and `failure` (or route `success` onward if this gets chained into another step later).

Starting the flow

Start both processors together. Since `QueryDatabaseTable` is stateful and tracks what it's already pulled, this should only need to run once for a full load — re-running it will only pick up new rows going forward.

Verifying the load

bash

```
hdfs dfs -ls /bronze/customers  
hdfs dfs -cat /bronze/customers/*
```

Confirms the customer records actually landed in the data lake before moving on to adjusting the controller service properties.

Adjust controller properties:

Controller Service Details

SETTINGS

PROPERTIES

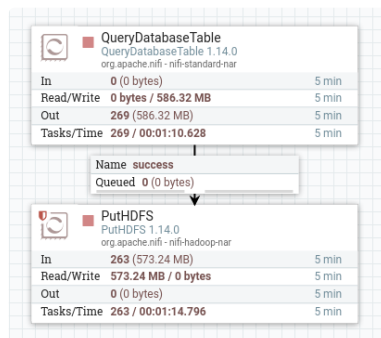
COMMENTS

Required field

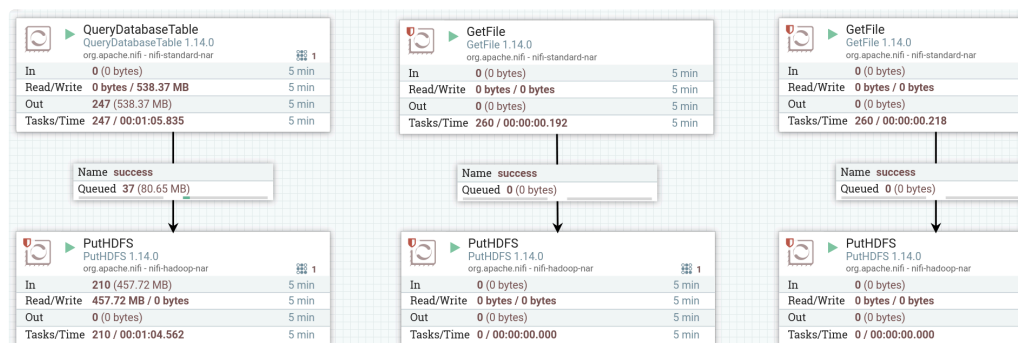
Property	Value
Database Connection URL	? jdbc:mariadb://localhost:3306/source_rdbms
Database Driver Class Name	? org.mariadb.jdbc.Driver
Database Driver Location(s)	? /usr/local/nifi/drivers/mariadb-java-client-3.1.4.jar
Kerberos Credentials Service	? No value set
Kerberos Principal	? No value set
Kerberos Password	? No value set
Database User	? root
Password	? No value set
Max Wait Time	? 500 millis
Max Total Connections	? 8
Validation query	? No value set
Minimum Idle Connections	? 0
Max Idle Connections	? 8
Max Connection Lifetime	? -1

OK

Streaming in action — RUN ONCE TO AVOID DROPPING AND CREATING FILES!!!



Final NiFi Processors:



Hive External table for bronze layer

First we open the hive shell:

```
hive
```

Database Initialization

```
CREATE DATABASE IF NOT EXISTS churn_bronze;
USE churn_bronze;
```

Tickets Table (CSV)

```
CREATE EXTERNAL TABLE IF NOT EXISTS bronze_tickets (
  Ticket_ID STRING,
  customer_id STRING,
  Customer_Name STRING,
  Customer_Email STRING,
  Ticket_Subject STRING,
  Ticket_Description STRING,
  Issue_Category STRING,
  Priority_Level STRING,
  Ticket_Channel STRING,
  Submission_Date STRING,
  Resolution_Time_Hours STRING,
  Assigned_Agent STRING,
  Satisfaction_Score STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
  "separatorChar" = ",",
  "quoteChar"     = "\""
)
STORED AS TEXTFILE
LOCATION '/bronze/tickets'
TBLPROPERTIES ("skip.header.line.count"="1");
```

Usage logs table (JSON)

```
CREATE EXTERNAL TABLE IF NOT EXISTS bronze_usage (
  usage_log_id STRING,
  `timestamp` STRING,
  customer_id STRING,
  source_system STRING,
  product_type STRING,
  monthly_balance DOUBLE,
  num_products INT
)
ROW FORMAT SERDE 'org.apache.hive.hcatalog.data.JsonSerDe'
STORED AS TEXTFILE
LOCATION '/bronze/usage';
```

Customers Table (Avro)

```
CREATE EXTERNAL TABLE IF NOT EXISTS bronze_customers (
  customer_id STRING,
  name STRING,
  email STRING,
  age STRING,
  gender STRING,
  geography STRING,
  is_active STRING,
  tenure_months STRING,
```

```
    date_opened STRING,  
    last_modified STRING  
  )  
  STORED AS AVRO  
  LOCATION '/bronze/customers';
```

Validation

```
SELECT * FROM bronze_tickets LIMIT 5;  
SELECT * FROM bronze_usage LIMIT 5;  
SELECT * FROM bronze_customers LIMIT 5;
```

Silver Layer

Create HDFS silver layer directories

```
[student@localhost ~]$ hdfs dfs -mkdir -p /silver/customers
[student@localhost ~]$ hdfs dfs -mkdir -p /silver/tickets
[student@localhost ~]$ hdfs dfs -mkdir -p /silver/usage_logs
```

Silver Layer with PySpark

Open jupyter notebook from the shell :

```
[student@localhost ~]$ pyspark
```

Select items to perform actions on them.

Upload New ↕

☐ 0	Project1	Name ▾	Last Modified	File size
	..		seconds ago	
☐	Datasets		an hour ago	
☐	Silver_Cleaning		6 minutes ago	

In the Home , navigate to the Project1 Directory and create `Silver_Cleaning` folder, this folder will contain the `jupyter` notebook for cleaning the data.

Select items to perform actions on them.

Upload New ↕

☐ 0	Project1 / Silver_Cleaning	Name ▾	Last Modified	File size
	..		seconds ago	
☐	 Silver_Layer_Cleaning_Data.ipynb		9 minutes ago	72 B

Cleaning Customers

Initializing SparkSession and inspecting customers.

```
In [5]: from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("SilverLayerTransformations") \
    .enableHiveSupport() \
    .getOrCreate()
```

```
In [10]: # This is for clean formatting when showing the dataframe

spark.conf.set("spark.sql.repl.eagerEval.enabled", True)
spark.conf.set("spark.sql.repl.eagerEval.maxNumRows", 10)
```

```
In [6]: spark.sql("SHOW DATABASES").show()

+-----+
| namespace|
+-----+
| churn_bronze|
| default|
| retail_db|
+-----+
```

```
In [7]: spark.sql("USE churn_bronze")

2026-09-03 00:47:55,662 WARN metastore.ObjectStore: Failed to get database global_temp, returning NoSuchObjectException
```

```
Out[7]: DataFrame[]
```

```
In [11]: df_customers = spark.read.format("avro").load("hdfs://localhost:9000/bronze/customers")

df_customers.printSchema()
df_customers
```

```
In [11]: df_customers = spark.read.format("avro").load("hdfs://localhost:9000/bronze/customers")

df_customers.printSchema()
df_customers

root
 |-- customer_id: string (nullable = true)
 |-- name: string (nullable = true)
 |-- email: string (nullable = true)
 |-- age: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- geography: string (nullable = true)
 |-- is_active: string (nullable = true)
 |-- tenure_months: string (nullable = true)
 |-- date_opened: string (nullable = true)
 |-- last_modified: string (nullable = true)
```

```
Out[11]:
```

customer_id	name	email	age	gender	geography	is_active	tenure_months	date_opened	last_modified
CUST-100000	Nicole Bernard	marissavang@examp...	37	Other	Cairo, EG	FALSE	22	2023-11-10	2024-06-18 04:45:...
CUST-100001	April Anderson	richard.brooks@en...	24	FEMALE	null	Y	59	2020/10/14	2024-07-15 20:25:...
CUST-100002	Karen Martinez	alexander03@examp...	72	F	null	True	44	2022-01-14	2025-02-01 23:33:...
CUST-100003	John Garza	KATHERINE40@EXAMP...	74	Other	Alexandria, EG	TRUE	47	11-Oct-2021	2025-04-24 06:34:...
CUST-100004	Tyler Lyons	ashley.mercer@com...	999	Other	Dubai, AE	no	16	2024/05/05	2024-07-18 08:34:...
CUST-100005	Timothy Rivera	sharon.morrison@e...	24	M	Toronto, CA	yes	20	2024-01-17	2025-08-19 11:19:...
CUST-100006	Andrew Cooper	david.fernandez@L...	74	FEMALE	Riyadh, SA	true	7	2025/02/14	2024-12-11 02:39:...
CUST-100007	Christopher Martin	henryraymond@exam...	26	Other	Riyadh, SA	TRUE	55	2021/02/04	2024-06-10 04:09:...
CUST-100008	Terry Williams	hodgeesean@examl...	25	Other	Riyadh, SA	TRUE	35	09/10/2022	2024-07-10 05:18:...
CUST-100009	Alexander Robertson	brady16@example.org	20	female	Sharqia, EG	Y	59	23/10/2020	2025-03-05 17:53:...

only showing top 10 rows

What the dirty-data output shows

Looking at the schema inspection, Spark has inferred several columns with the wrong type because the Bronze data mixes text and numbers. The `is_active` column, which should hold a simple true/false flag, was read as a `string` — and when we look at the actual rows we can see why: the raw values are not uniform. Some rows contain `"true"` or `"false"`, others use `"yes"` / `"no"`, and a few even store `"1"` / `"0"` or `"active"` / `"inactive"`. Because a single type can't represent all of these variants, Spark defaulted to `string` rather than recognising the Boolean intent. The same problem appears with the numeric fields: `age` and `tenure_months` were inferred as `double` because of stray decimal points or mixed formatting, but both should be whole integers that hold an age and a month count.

Beyond the schema, the data display may reveal two content-quality problems. First, duplicate `customer_id` values — the same customer may appear more than once, which would break any downstream join or aggregation that relies on a unique key. Second, the `name` column is inconsistent in casing: some entries are entirely uppercase like `"JOHN DOE"`, others are all lowercase

like "john doe", and others are mixed like "John doe". The same person's name can appear in different forms, which makes grouping, searching, and reporting on customers unreliable. To fix this we need to cast every field to its correct type, keep only one row per customer, and standardise the names so they all read the same way.

The age column needs its own validation on top of the type cast. Because age was read in as a raw string, the source data holds some entries that are null and others that fall outside any realistic adult range — values below 18 or above 100 are almost certainly data-entry errors or placeholder junk rather than a real customer age. A clean silver layer should not keep these, so we drop any record whose age is null and any whose age sits outside the 18–100 window, leaving only rows with believable, usable ages.

The is_active handling also goes further than a plain cast: rather than assuming every row already reads exactly "true" or "false", the clean-up maps each of the many observed spellings to a single Boolean, so a stray "yes", "1", "active" or similar variant is normalised correctly instead of being dropped or left ambiguous.

The geography field has the same kind of messiness that the identifiers do. Some cells are genuinely null, while others do not store a real value at all — they hold the literal SQL null sentinel \N, or are blank strings that just look empty but are still treated as data. Any of these cases is not a usable location, so they are all collapsed into a single "Unknown" marker, and the remaining values are trimmed of surrounding whitespace so they compare cleanly.

The gender column is normalised too, because the raw rows spell it inconsistently: "female" and "f" both become "Female", "male" and "m" both become "Male", "other" maps to "Other" as written, and anything unrecognisable — including nulls — falls back to "Unknown" so the column ends up with one consistent set of values instead of a jumble of abbreviations and variants.

Code :

```
from pyspark.sql import functions as F
from pyspark.sql.types import IntegerType, BooleanType, TimestampType

# 1. Clean and Transform Customers Bronze Data
customers_silver = (
    df_customers
    .filter(
        F.col("customer_id").isNotNull() &
        (F.trim(F.col("customer_id")) != "")
    )
    # Trim identifiers and standard text fields
    .withColumn("customer_id", F.trim(F.col("customer_id")))

    .withColumn(
        "name",
        F.initcap(F.coalesce(F.trim(F.col("name")), F.lit("Unknown")))
    )

    .withColumn("email", F.coalesce(F.lower(F.trim(F.col("email"))), F.lit("unknown@domain.com")))

    # Cast age and DROP rows with NULL or invalid values (< 18 or > 100)
    .withColumn("age", F.col("age").cast(IntegerType()))
    .filter(
        F.col("age").isNotNull() &
        (F.col("age") >= 18) &
        (F.col("age") <= 100)
    )

    # Normalize gender to standard values (Fills nulls with 'Unknown')
    .withColumn(
        "gender",
        F.when(F.lower(F.col("gender")).isin("female", "f"), "Female")
        .when(F.lower(F.col("gender")).isin("male", "m"), "Male")
        .when(F.lower(F.col("gender")) == "other", "Other")
        .otherwise("Unknown")
    )
)
```

```

)

# Standardize geography (Replaces SQL null strings like '\N' and blank values with 'Unknown')
.withColumn(
    "geography",
    F.when(
        F.col("geography").isNull() |
        (F.col("geography") == "\\N") |
        (F.trim(F.col("geography")) == ""),
        "Unknown"
    ).otherwise(F.trim(F.col("geography")))
)

# Cast is_active string flags to True/False Boolean
.withColumn(
    "is_active",
    F.when(F.lower(F.col("is_active")).isin("true", "t", "yes", "y", "1", "active"), True)
    .when(F.lower(F.col("is_active")).isin("false", "f", "no", "n", "0", "inactive"), False)
    .otherwise(False)
    .cast(BooleanType())
)

# Cast numeric metrics, defaulting NULL tenure to 0
.withColumn("tenure_months", F.coalesce(F.col("tenure_months").cast(IntegerType()), F.lit(0)))
.filter(
    F.col("tenure_months") >= 0
)

# Parse mixed date string formats into standard DateType (YYYY-MM-DD)
.withColumn(
    "date_opened",
    F.coalesce(
        F.to_date(F.col("date_opened"), "yyyy-MM-dd"),
        F.to_date(F.col("date_opened"), "yyyy/MM/dd"),
        F.to_date(F.col("date_opened"), "dd-MMM-yyyy"),
        F.to_date(F.col("date_opened"), "dd/MM/yyyy"),
        F.to_date(F.col("date_opened"), "MM-dd-yyyy"),
    )
)

# Drop unknown date opened
.filter(F.col("date_opened").isNotNull())

# Cast audit timestamp and dropping unknown dates
.withColumn("last_modified", F.col("last_modified").cast(TimestampType()))
.filter(F.col("last_modified").isNotNull())

.dropDuplicates(["customer_id"])
)

```

Results :

```
In [28]: customers_silver.printSchema()
customers_silver

root
|-- customer_id: string (nullable = true)
|-- name: string (nullable = false)
|-- email: string (nullable = false)
|-- age: integer (nullable = true)
|-- gender: string (nullable = false)
|-- geography: string (nullable = true)
|-- is_active: boolean (nullable = false)
|-- tenure_months: integer (nullable = false)
|-- date_opened: date (nullable = true)
|-- last_modified: timestamp (nullable = true)
```

```
Out[28]:
```

customer_id	name	email	age	gender	geography	is_active	tenure_months	date_opened	last_modified
CUST-100339	Rhonda Webb	tom.bennett@enter...	36	Unknown	Paris, FR	true	42	2022-03-10	2025-05-31 12:34:12
CUST-100675	Scott Perez	monica.martinez@e...	49	Unknown	Toronto, CA	false	24	2023-09-17	2024-07-11 08:28:14
CUST-100712	Steven Baker	megan91@example.net	71	Female	Sharqia, EG	true	4	2025-05-17	2024-07-25 18:53:51
CUST-100905	Mr. Scott Daniels	brian.morris@ente...	57	Unknown	New York, US	true	27	2023-06-29	2025-08-08 17:22:02
CUST-101195	Mr. Tyler Rose Jr.	kelly.campbell@te...	18	Unknown	Giza, EG	true	12	2024-09-01	2024-05-03 14:14:50
CUST-101670	John Moss	crystal.hickman@t...	74	Male	Dubai, AE	false	35	2022-10-25	2024-10-24 11:28:44
CUST-102235	Chad Jones	ryandebbie@exampl...	64	Male	Mumbai, IN	true	37	2022-08-03	2025-08-18 08:46:31
CUST-102350	Leonard Medina	zboyd@example.net	35	Female	Mumbai, IN	true	33	2022-12-09	2025-08-30 23:37:49
CUST-102523	Keith Payne	aleon@example.org	21	Other	New York, US	true	57	2020-12-23	2025-07-05 14:15:37

Cleaning Tickets

```
In [32]: # Tickets

df_tickets = (
    spark.read
    .format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("hdfs://localhost:9000/bronze/tickets")
)

# Print schema and row count
df_tickets.printSchema()
print("Total Ticket Records:", df_tickets.count())

# Inspect null counts per column
df_tickets.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c)
    for c in df_tickets.columns
]).show()

root
|-- Ticket ID: string (nullable = true)
|-- customer id: string (nullable = true)
|-- Customer Name: string (nullable = true)
|-- Customer Email: string (nullable = true)
|-- Ticket Subject: string (nullable = true)
|-- Ticket Description: string (nullable = true)
|-- Issue Category: string (nullable = true)
|-- Priority Level: string (nullable = true)
|-- Ticket Channel: string (nullable = true)
|-- Submission Date: string (nullable = true)
|-- Resolution Time Hours: integer (nullable = true)
|-- Assigned Agent: string (nullable = true)
|-- Satisfaction Score: integer (nullable = true)

Total Ticket Records: 20000

+-----+-----+-----+-----+-----+-----+-----+-----+
|Ticket ID|customer id|Customer Name|Customer Email|Ticket Subject|Ticket Description|Issue Category|Priority_Level|
+-----+-----+-----+-----+-----+-----+-----+-----+
|0|0|0|0|0|0|0|0|
+-----+-----+-----+-----+-----+-----+-----+-----+
|37|0|0|0|557|592|600|0|6|
+-----+-----+-----+-----+-----+-----+-----+-----+
```

What the dirty-data output shows

The tickets data has a structural problem right away: the column names are in PascalCase rather than lowercase `snake_case` — `Ticket_ID`, `Customer_Name`, `Submission_Date`, `Assigned_Agent`, and so on.

Two numeric columns need attention as well. `Satisfaction_Score` was inferred as a `double` and, on inspection, some rows might fall outside the valid 1–5 range or are completely null, so those need to be validated and dropped.

`Resolution_Time_Hours` also contains nulls that would break any later calculation, so those rows have to go too.

The `Ticket_Description` field is another concern: It stores the valid description but after some point it stores gibberish sentences that has no any relation with the actual sentence so i removed the extra sentence from each description.

The identifier column `Ticket_ID` itself needs cleaning before it can be trusted as a primary key: some cells are null. The same applies to the `customer_id` link — any record that is missing its `Ticket_ID` or its `customer_id` cannot be joined back to the rest of the warehouse, so these rows are dropped up front, and the surviving values are trimmed of whitespace. The email address `customer_email` is also treated for consistency: it is lowercased and trimmed so that the same customer's address always compares equal, and the `customer_name` field is run through `initcap` like the customer names in the previous section so it reads the same everywhere.

The remaining text columns each get a targeted standardisation rather than a generic clean-up. `issue_category`, `priority_level`, `ticket_channel`, and `assigned_agent` are all run through `initcap` to fix inconsistent casing, and any null `priority_level` is defaulted to `"Medium"` rather than left as a gap in the data, since Medium is the sensible default for a ticket that was never given an explicit priority.

A null `assigned_agent` is similarly replaced with `"Unknown"` so the column never contains empty cells. The `submission_date` behaves a little differently: it is stored as a raw string that can be null or contain stray whitespace, and because it does not parse reliably as a real timestamp, it is trimmed and defaulted to the string `"Unknown"` when empty — keeping the row but marking the date as not provided instead of guessing. Finally, `ticket_description` is cut down to its first sentence, because the raw entries run on for many sentences and only the opening is meaningful for a concise silver layer.

Code :

```
from pyspark.sql.types import DoubleType
```

```

tickets_silver = (
    df_tickets
    # 1. Drop records missing primary keys or customer links
    .filter(
        F.col("Ticket_ID").isNotNull() & (F.trim(F.col("Ticket_ID")) != "") &
        F.col("customer_id").isNotNull() & (F.trim(F.col("customer_id")) != "")
    )

    # 2. Standardize column names to lowercase snake_case
    .withColumnRenamed("Ticket_ID", "ticket_id")
    .withColumnRenamed("Customer_Name", "customer_name")
    .withColumnRenamed("Customer_Email", "customer_email")
    .withColumnRenamed("Ticket_Subject", "ticket_subject")
    .withColumnRenamed("Ticket_Description", "ticket_description")
    .withColumnRenamed("Issue_Category", "issue_category")
    .withColumnRenamed("Priority_Level", "priority_level")
    .withColumnRenamed("Ticket_Channel", "ticket_channel")
    .withColumnRenamed("Submission_Date", "submission_date")
    .withColumnRenamed("Resolution_Time_Hours", "resolution_time_hours")
    .withColumnRenamed("Assigned_Agent", "assigned_agent")
    .withColumnRenamed("Satisfaction_Score", "satisfaction_score")

    # 3. Trim IDs and standardize text fields
    .withColumn("ticket_id", F.trim(F.col("ticket_id")))
    .withColumn("customer_id", F.trim(F.col("customer_id")))
    .withColumn("customer_name", F.initcap(F.trim(F.col("customer_name"))))
    .withColumn("customer_email", F.lower(F.trim(F.col("customer_email"))))

    # Extract original description up to the first '.' or '?'
    .withColumn(
        "ticket_description",
        F.regexp_extract(F.col("ticket_description"), r"^(.*?[.?])", 1)
    )

    .withColumn("issue_category", F.initcap(F.trim(F.col("issue_category"))))
    .withColumn("priority_level", F.initcap(F.trim(F.col("priority_level"))))
    .withColumn("ticket_channel", F.initcap(F.trim(F.col("ticket_channel"))))
    .withColumn("assigned_agent", F.initcap(F.trim(F.col("assigned_agent"))))

    # Make null priority levels as Medium (the default priority)
    .withColumn("priority_level",
        F.coalesce(F.col("priority_level"), F.lit("Medium"))
    )

    # 4. Handle Submission Date (Default to 'Unknown' string if null)
    .withColumn(
        "submission_date",
        F.coalesce(F.trim(F.col("submission_date")), F.lit("Unknown"))
    )

    # 5. Handle null Assigned Agents
    .withColumn(
        "assigned_agent",
        F.coalesce(F.col("assigned_agent"), F.lit("Unknown"))
    )

    # 6. Type Casting for Metrics
    .withColumn("resolution_time_hours", F.col("resolution_time_hours").cast(DoubleType()))
    .withColumn("satisfaction_score", F.col("satisfaction_score").cast(IntegerType()))

```

```

# 7. DROP ROWS where satisfaction_score is null or outside 1-5
.filter(
  F.col("satisfaction_score").isNotNull() &
  (F.col("satisfaction_score") >= 1) &
  (F.col("satisfaction_score") <= 5)
)

# 8. DROP ROWS where resolution_time_hours is null
.filter(F.col("resolution_time_hours").isNotNull())

# Deduplicate on ticket primary key
.dropDuplicates(["ticket_id"])
)

```

```

In [56]: # Inspect null counts per column after cleaning
tickets_silver.select(
  F.count(F.when(F.col(c).isNull(), c)).alias(c)
  for c in tickets_silver.columns
).show()

[Stage 61:=====> (164 + 10) / 200]

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|ticket_id|customer_id|customer_name|customer_email|ticket_subject|ticket_description|issue_category|priority_level|ticket_channel|submission_date|resolution_time_hours|assigned_agent|satisfaction_score|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|0|0|0|0|0|0|0|0|0|0|0|0|0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Cleaning Usage_Logs

```

In [63]: # Usage_Logs
df_ologs = (
  spark.read
    .format("json")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("hdfs://localhost:9000/bronze/usage")
)

# Print schema and row count
df_ologs.printSchema()
print("Total Usage Log Records:", df_tickets.count())

# Inspect null counts per column
df_ologs.select(
  F.count(F.when(F.col(c).isNull(), c)).alias(c)
  for c in df_ologs.columns
).show()

root
|-- customer_id: string (nullable = true)
|-- monthly_balance: string (nullable = true)
|-- num_products: long (nullable = true)
|-- product_type: string (nullable = true)
|-- source_system: string (nullable = true)
|-- timestamp: string (nullable = true)
|-- usage_log_id: string (nullable = true)

Total Usage Log Records: 20000
+-----+-----+-----+-----+-----+-----+
|customer_id|monthly_balance|num_products|product_type|source_system|timestamp|usage_log_id|
+-----+-----+-----+-----+-----+-----+
|897|958|901|868|611|0|0|
+-----+-----+-----+-----+-----+-----+

```

What the dirty-data output shows

The dirty data follows the same pattern when cleaning it in the previous datasets, `customer_id` must be trimmed, null dropped as well as the `usage_log_id` + deduplication of `usage_log_id` if any (There is no duplicate `usage_log_id` in this dataset but i included this cleaning approach to account for further duplicate data)

`monthly_balance` needed to be casted as `double` and not `string` and also validated against invalid values like nulls and zeros.
`num_products` needed to be casted to `int` and not `long` to save memory space, also validated against null and zeros

Text categories needed to be standardized.

Code :

```
from pyspark.sql import functions as F
from pyspark.sql.types import IntegerType, DoubleType, TimestampType

usage_logs_silver = (
    df_uloogs
    # 1. Drop rows missing primary key or customer link
    .filter(
        F.col("usage_log_id").isNotNull() & (F.trim(F.col("usage_log_id")) != "") &
        F.col("customer_id").isNotNull() & (F.trim(F.col("customer_id")) != "")
    )

    # 2. Trim identifiers
    .withColumn("usage_log_id", F.trim(F.col("usage_log_id")))
    .withColumn("customer_id", F.trim(F.col("customer_id")))

    # 3. Clean and cast numeric metrics
    .withColumn("monthly_balance", F.col("monthly_balance").cast(DoubleType()))
    .withColumn("num_products", F.col("num_products").cast(IntegerType()))

    # Filter out invalid negative balances or product counts
    .filter(
        (F.col("monthly_balance") >= 0) &
        (F.col("num_products") >= 0)
    )

    # 4. Standardize text categories
    .withColumn(
        "product_type",
        F.initcap(F.coalesce(F.trim(F.col("product_type")), F.lit("Unknown")))
    )
    .withColumn(
        "source_system",
        F.upper(F.coalesce(F.trim(F.col("source_system")), F.lit("UNKNOWN")))
    )

    # 5. Parse timestamp and drop missing/unparseable timestamps
    .withColumn(
        "timestamp",
        F.coalesce(
            F.col("timestamp").cast(TimestampType()),
            F.to_timestamp(F.col("timestamp"), "yyyy-MM-dd HH:mm:ss"),
            F.to_timestamp(F.col("timestamp"), "yyyy-MM-dd'T'HH:mm:ss")
        )
    )
    .filter(F.col("timestamp").isNotNull())

    # 6. Deduplicate by unique log ID
    .dropDuplicates(["usage_log_id"])
)
```


Sample Output of cleaned tickets

```
In [71]: usage_logs_silver.select([
        F.count(F.when(F.col(c).isNull(), c)).alias(c)
        for c in usage_logs_silver.columns
    ]).show()
```

usage_logs_silver

```
+-----+-----+-----+-----+-----+-----+-----+
|customer_id|monthly_balance|num_products|product_type|source_system|timestamp|usage_log_id|
+-----+-----+-----+-----+-----+-----+-----+
|          0|              0|          0|          0|          0|          0|          0|
+-----+-----+-----+-----+-----+-----+-----+
```

```
Out[71]:
```

customer_id	monthly_balance	num_products	product_type	source_system	timestamp	usage_log_id
CUST-100014	45375.4	3	Credit Card	CORE_BANKING	2025-04-23 20:35:...	USG-300011
CUST-100163	13604.5	1	Investment Fund	CORE_BANKING	2026-02-11 06:18:...	USG-300261
CUST-100388	37550.75	4	Mortgage	MOBILE_APP	2026-04-15 03:46:...	USG-300629
CUST-100418	19143.45	1	Savings Account	WEB_PORTAL	2026-07-30 23:46:...	USG-300674
CUST-100471	22741.99	3	Investment Fund	MOBILE_APP	2025-08-10 05:17:...	USG-300753
CUST-100503	21528.35	1	Savings Account	MOBILE_APP	2025-09-22 18:30:...	USG-300801
CUST-101217	46825.33	4	Personal Loan	WEB_PORTAL	2025-11-23 08:19:...	USG-301913
CUST-101271	5653.28	1	Savings Account	MOBILE_APP	2025-10-01 18:32:...	USG-302003
CUST-101663	38222.65	5	Savings Account	MOBILE_APP	2026-04-22 20:59:...	USG-302630
CUST-101784	10413.86	5	Checking Account	CORE_BANKING	2026-05-30 15:58:...	USG-302823
CUST-101872	37885.26	1	Mortgage	CORE_BANKING	2026-01-13 07:29:...	USG-302947
CUST-101913	19447.76	3	Savings Account	CORE_BANKING	2026-08-15 08:16:...	USG-303008

Writing the cleaned data to HDFS

```
In [74]: # Write customers_silver
(
    customers_silver.coalesce(1)
    .write
    .mode("overwrite")
    .format("parquet")
    .save("hdfs://localhost:9000/silver/customers")
)

# Write tickets_silver
(
    tickets_silver.coalesce(1)
    .write
    .mode("overwrite")
    .format("parquet")
    .save("hdfs://localhost:9000/silver/tickets")
)

# Write usage_logs_silver
(
    usage_logs_silver.coalesce(1)
    .write
    .mode("overwrite")
    .format("parquet")
    .save("hdfs://localhost:9000/silver/usage_logs")
)
```

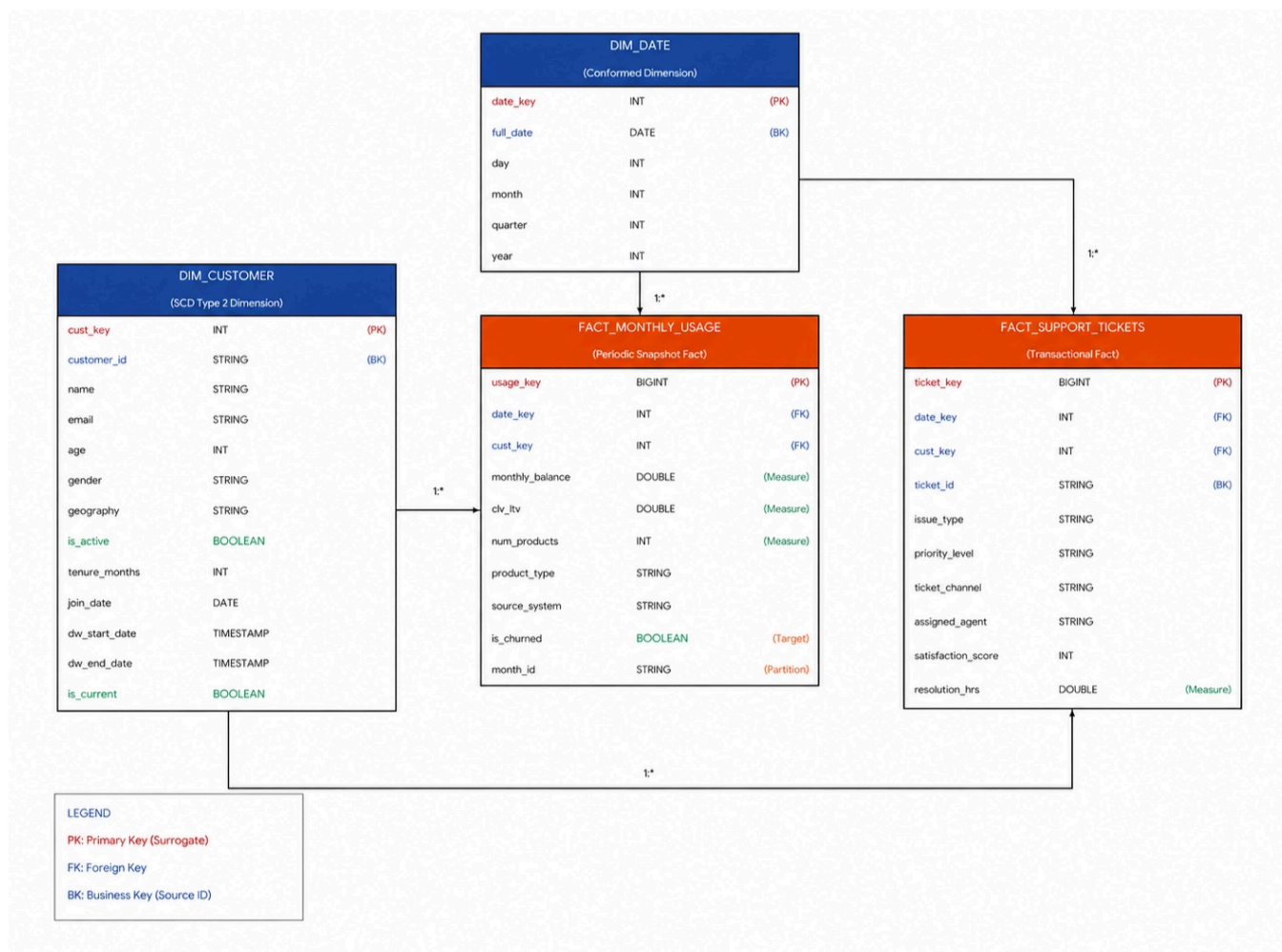
```
In [ ]:
```

Verifying

```
student : bash - Konsole
File Edit View Bookmarks Settings Help
student@localhost ~]$ hdfs dfs -ls /silver/usage_logs
Found 2 items
-rw-r--r-- 1 student supergroup          0 2026-09-03 06:15 /silver/usage_logs/ SUCCESS
-rw-r--r-- 1 student supergroup    797031 2026-09-03 06:15 /silver/usage_logs/part-00000-9b5e0dd4-373f-403d-91cf-1efe289c9cb2-c000.snappy.parquet
student@localhost ~]$ hdfs dfs -ls /silver/customers
Found 2 items
-rw-r--r-- 1 student supergroup          0 2026-09-03 06:15 /silver/customers/ SUCCESS
-rw-r--r-- 1 student supergroup    744621 2026-09-03 06:15 /silver/customers/part-00000-0915a94a-1661-48f2-a8bf-1d16bfb44c9b-c000.snappy.parquet
student@localhost ~]$ hdfs dfs -ls /silver/tickets
Found 2 items
-rw-r--r-- 1 student supergroup          0 2026-09-03 06:15 /silver/tickets/ SUCCESS
-rw-r--r-- 1 student supergroup    840077 2026-09-03 06:15 /silver/tickets/part-00000-b31c7823-e0ac-4da1-a462-c9f3eebfbcb69-c000.snappy.parquet
student@localhost ~]$
```

We can see here that the files are thrown to hdfs as parquet with one partition, why one partition? because the file sizes are small (less than 1 GB) so there is no need for partitioning.

Gold Layer



Schema Type

This architecture is a **Galaxy Schema** (also known as a Fact Constellation Schema). While it relies on core star schema principles, it utilizes multiple fact tables (`FACT_MONTHLY_USAGE` and `FACT_SUPPORT_TICKETS`) that share conformed dimensions. A traditional star schema features only a single fact table at the center.

Multiple Fact Tables, Grains, & Measures

Isolating metrics into separate fact tables prevents granularity mismatches and data distortion, ensuring high-frequency event data does not bloat regular periodic summaries.

- **FACT_MONTHLY_USAGE** (Periodic Snapshot Fact)
 - **Grain:** One row per customer per month.
 - **Purpose:** Captures aggregated monthly balances and product counts at regular intervals, serving directly as the feature matrix for customer churn prediction models.
 - **Measures:**
 - `monthly_balance` is a **semi-additive measure**; it can be logically summed across different customers to find total bank holdings for a specific month, but it cannot be logically summed across time (adding January's balance to February's balance artificially inflates the actual funds).
 - `clv_ltv` (Customer Lifetime Value) is a **semi-additive measure** calculated as a rolling historical total. It can be aggregated across customer segments (e.g., total CLV of all active users), but it cannot be summed across time dimensions, as a customer's February CLV inherently already includes their January CLV.
- **FACT_SUPPORT_TICKETS** (Transactional Fact)
 - **Grain:** One row per support ticket.
 - **Purpose:** Records individual service events exactly when they happen, keeping operational support tracking granular without distorting monthly periodic snapshots.
 - **Measures:**

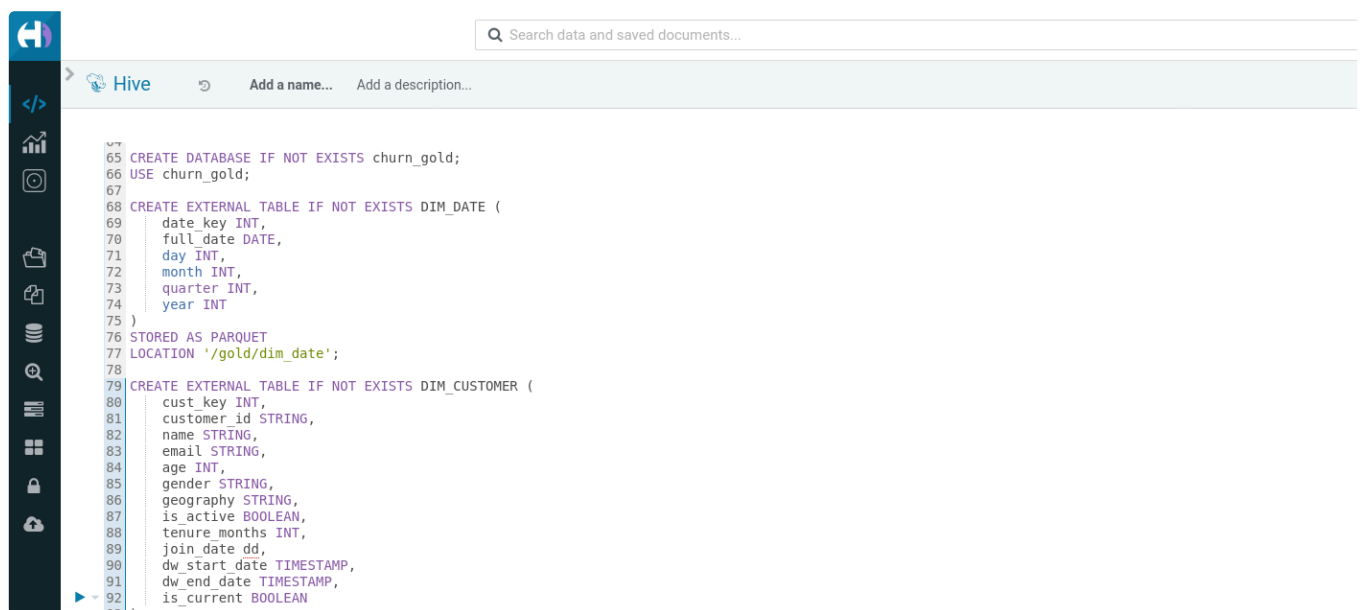
- `resolution_hrs` is an **additive measure**; it can be summed across any dimension (time, customer, agent) to calculate total time spent on support.
- `satisfaction_score` is a **non-additive measure**; it cannot be logically summed anywhere and must be aggregated using mathematical averages.

Conformed Dimension Tables

- **DIM_CUSTOMER**: Tracks user demographic profiles, active status, and geographic regions using a Hybrid Slowly Changing Dimension (SCD) model. It utilizes a system-generated surrogate key (`cust_key`) to insulate the data warehouse against upstream source system changes and tracks historical state changes to prevent data leakage during machine learning training.
- **DIM_DATE**: A static, pre-calculated calendar reference table using an integer surrogate key (`date_key`). It centralizes all date-math (day, month, quarter, year), preventing expensive runtime date conversions and parsing across billions of fact rows during analytical queries.

1. Hive Data Definition Language (DDL)

Execute these statements in your Hive or Spark SQL environment to create the Gold layer schema.



```

65 CREATE DATABASE IF NOT EXISTS churn_gold;
66 USE churn_gold;
67
68 CREATE EXTERNAL TABLE IF NOT EXISTS DIM_DATE (
69     date_key INT,
70     full_date DATE,
71     day INT,
72     month INT,
73     quarter INT,
74     year INT
75 )
76 STORED AS PARQUET
77 LOCATION '/gold/dim_date';
78
79 CREATE EXTERNAL TABLE IF NOT EXISTS DIM_CUSTOMER (
80     cust_key INT,
81     customer_id STRING,
82     name STRING,
83     email STRING,
84     age INT,
85     gender STRING,
86     geography STRING,
87     is_active BOOLEAN,
88     tenure_months INT,
89     join_date dd,
90     dw_start_date TIMESTAMP,
91     dw_end_date TIMESTAMP,
92     is_current BOOLEAN
93 )
  
```

```

CREATE DATABASE IF NOT EXISTS churn_gold;
USE churn_gold;

CREATE EXTERNAL TABLE IF NOT EXISTS DIM_DATE (
    date_key INT,
    full_date DATE,
    day INT,
    month INT,
    quarter INT,
    year INT
)
STORED AS PARQUET
LOCATION '/gold/dim_date';
  
```

```

CREATE EXTERNAL TABLE IF NOT EXISTS DIM_CUSTOMER (
    cust_key INT,
    customer_id STRING,
    name STRING,
    email STRING,
    age INT,
    gender STRING,
    geography STRING,
  
```

```

    is_active BOOLEAN,
    tenure_months INT,
    join_date DATE,
    dw_start_date TIMESTAMP,
    dw_end_date TIMESTAMP,
    is_current BOOLEAN
)
STORED AS PARQUET
LOCATION '/gold/dim_customer';

CREATE EXTERNAL TABLE IF NOT EXISTS FACT_SUPPORT_TICKETS (
    ticket_key BIGINT,
    date_key INT,
    cust_key INT,
    ticket_id STRING,
    issue_type STRING,
    priority_level STRING,
    ticket_channel STRING,
    assigned_agent STRING,
    satisfaction_score INT,
    resolution_hrs DOUBLE
)
STORED AS PARQUET
LOCATION '/gold/fact_support_tickets';

CREATE EXTERNAL TABLE IF NOT EXISTS FACT_MONTHLY_USAGE (
    usage_key BIGINT,
    date_key INT,
    cust_key INT,
    monthly_balance DOUBLE,
    clv_ltv DOUBLE,
    num_products INT,
    product_type STRING,
    source_system STRING,
    is_churned BOOLEAN
)
PARTITIONED BY (month_id STRING)
STORED AS PARQUET
LOCATION '/gold/fact_monthly_usage';

```

2. PySpark Hybrid ETL Pipeline

2.1 Session Initialization & Ingestion

Instantiates a Spark Session with `.enableHiveSupport()` to allow interaction with the Hive metastore for persistent table creation and metadata management. It lazily reads columnar Parquet files from the Silver layer.

```

from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder \
    .appName("Gold_Layer_Hybrid_SCD_Pipeline") \
    .enableHiveSupport() \
    .getOrCreate()

# 1. Read Silver Layer Parquet Data
df_cust_silver = spark.read.parquet("/silver/customers")

```

```
df_usage_silver = spark.read.parquet("/silver/usage_logs")
df_ticket_silver = spark.read.parquet("/silver/tickets")
```

2.2 Date Dimension (`DIM_DATE`) Generation

Generates a continuous daily calendar from `2020-01-01` to `2026-12-31` using Spark SQL `sequence()` and `explode()` functions. It pre-calculates date attributes and assigns an integer surrogate key (`yyyyMMdd`) to enable highly optimized integer-based joins.

```
# 2. Populate Master Date Dimension
dim_date = spark.sql("""
    SELECT
        CAST(date_format(calendar_date, 'yyyyMMdd') AS INT) AS date_key,
        CAST(calendar_date AS DATE) AS full_date,
        day(calendar_date) AS day,
        month(calendar_date) AS month,
        quarter(calendar_date) AS quarter,
        year(calendar_date) AS year
    FROM (
        SELECT explode(sequence(to_date('2020-01-01'), to_date('2026-12-31'), interval 1 day)) AS
        calendar_date
    )
""")

dim_date.write.mode("overwrite").option("path", "/gold/dim_date").saveAsTable("churn_gold.DIM_DATE")
```

2.3 Customer Dimension (`DIM_CUSTOMER`) Hybrid SCD Engine

- **Lineage Breaking:** Uses `localCheckpoint(eager=True)` to cut the Spark Direct Acyclic Graph (DAG) lineage, preventing `AnalysisException` cyclic reference errors when overwriting the same table.
- **SCD Type 2 (Historical Versioning):** Triggered by predictive feature changes (`age`, `geography`, `is_active`). Retires the old record by capping `dw_end_date` and inserts a new active row.
- **SCD Type 1 (Inline Overwrite):** Triggered by non-predictive administrative updates (`name`, `email`, `gender`, `tenure_months`). Overwrites values on the active record directly without generating a new row.

```
# 3. HYBRID SCD1 & SCD2 MERGE FOR DIM_CUSTOMER
# =====
dim_cust_table_name = "churn_gold.DIM_CUSTOMER"

incoming_cust = df_cust_silver.select(
    F.col("customer_id"),
    F.col("name"),
    F.col("email"),
    F.col("age"),
    F.col("gender"),
    F.col("geography"),
    F.col("is_active"),
    F.col("tenure_months"),
    F.to_date(F.col("date_opened")).alias("join_date"),
    F.to_timestamp(F.col("last_modified")).alias("last_modified")
)

# Version-proof check for table existence
table_exists = False
try:
    # Checkpoint immediately breaks the read lineage so we can safely overwrite later
    existing_dim = spark.table(dim_cust_table_name).localCheckpoint(eager=True)
    table_exists = True
except Exception:
```

pass

```
if table_exists:
    max_key = existing_dim.agg(F.max("cust_key")).collect()[0][0] or 0

    active_dim = existing_dim.filter(F.col("is_current") == True)
    historical_dim = existing_dim.filter(F.col("is_current") == False)

    joined = active_dim.alias("dim").join(
        incoming_cust.alias("inc"),
        F.col("dim.customer_id") == F.col("inc.customer_id"),
        how="full_outer"
    )

    # SCD2 triggers: age, geography, is_active
    scd2_cond = (F.col("inc.age") != F.col("dim.age")) | \
        (F.col("inc.geography") != F.col("dim.geography")) | \
        (F.col("inc.is_active") != F.col("dim.is_active"))

    # SCD1 triggers: name, email, gender, tenure_months
    scd1_cond = (F.col("inc.name") != F.col("dim.name")) | \
        (F.col("inc.email") != F.col("dim.email")) | \
        (F.col("inc.gender") != F.col("dim.gender")) | \
        (F.col("inc.tenure_months") != F.col("dim.tenure_months"))

    unchanged = joined.filter(~scd2_cond & ~scd1_cond &
        F.col("inc.customer_id").isNotNull()).select("dim.*")

    scd1_updates = joined.filter(~scd2_cond & scd1_cond).select(
        F.col("dim.cust_key"), F.col("dim.customer_id"),
        F.col("inc.name"), F.col("inc.email"),
        F.col("dim.age"), F.col("inc.gender"), F.col("dim.geography"), F.col("dim.is_active"),
        F.col("inc.tenure_months"), F.col("dim.join_date"), F.col("dim.dw_start_date"),
        F.col("dim.dw_end_date"), F.col("dim.is_current")
    )

    expired = joined.filter(scd2_cond).select(
        F.col("dim.cust_key"), F.col("dim.customer_id"), F.col("dim.name"), F.col("dim.email"),
        F.col("dim.age"), F.col("dim.gender"), F.col("dim.geography"), F.col("dim.is_active"),
        F.col("dim.tenure_months"), F.col("dim.join_date"), F.col("dim.dw_start_date"),
        F.col("inc.last_modified").alias("dw_end_date"), F.lit(False).alias("is_current")
    )

    new_records_source = joined.filter(F.col("dim.customer_id").isNull() | scd2_cond).select("inc.*")
    insert_window = Window.orderBy("customer_id")
    new_inserts = new_records_source.withColumn("cust_key", max_key +
        F.row_number().over(insert_window)).select(
        "cust_key", "customer_id", "name", "email", "age", "gender", "geography", "is_active",
        "tenure_months", "join_date", F.col("last_modified").alias("dw_start_date"),
        F.to_timestamp(F.lit("9999-12-31 23:59:59")).alias("dw_end_date"),
        F.lit(True).alias("is_current")
    )

    missing = joined.filter(F.col("inc.customer_id").isNull()).select("dim.*")

    final_dim_customer = historical_dim.unionByName(unchanged) \
        .unionByName(scd1_updates).unionByName(expired).unionByName(new_inserts).unionByName(missing)
else:
    insert_window = Window.orderBy("customer_id")
```

```

final_dim_customer = incoming_cust.withColumn("cust_key", F.row_number().over(insert_window)).select(
    "cust_key", "customer_id", "name", "email", "age", "gender", "geography", "is_active",
    "tenure_months", "join_date", F.col("last_modified").alias("dw_start_date"),
    F.to_timestamp(F.lit("9999-12-31 23:59:59")).alias("dw_end_date"),
    F.lit(True).alias("is_current")
)
final_dim_customer = final_dim_customer.localCheckpoint(eager=True)

final_dim_customer.write.mode("overwrite").option("path",
"/gold/dim_customer").saveAsTable(dim_cust_table_name)

```

2.4 Fact Table Processing & Point-in-Time Joins

- **Customer Lifetime Value (CLV):** Uses a Spark `Window` function to calculate a running cumulative total of `monthly_balance`.
- **Data Leakage Prevention:** Fact tables join to `DIM_CUSTOMER` using strict temporal bounds (`snapshot_time >= dw_start_date AND < dw_end_date`) to link events to the exact historical state the customer was in.
- **Target Variable:** Calculates the ML target label (`is_churned`) dynamically by evaluating the customer's active status against their termination date.

```

# Cache active dimension for fast fact table joins
dim_cust_df = spark.table(dim_cust_table_name)
dim_date_df = spark.table("churn_gold.DIM_DATE")

# 1. Process Monthly Usage Fact
usage_monthly = df_usage_silver \
    .withColumn("month_id", F.date_format("timestamp", "yyyy-MM")) \
    .withColumn("snapshot_date", F.last_day(F.to_date("timestamp"))) \
    .withColumn("snapshot_time", F.to_timestamp("timestamp")) \
    .groupBy("customer_id", "month_id", "snapshot_date", "snapshot_time", "product_type",
"source_system") \
    .agg(F.sum("monthly_balance").alias("monthly_balance"), F.max("num_products").alias("num_products"))

clv_window = Window.partitionBy("customer_id").orderBy("month_id").rowsBetween(Window.unboundedPreceding,
Window.currentRow)

fact_monthly_usage = usage_monthly \
    .withColumn("clv_ltv", F.sum("monthly_balance").over(clv_window)) \
    .join(dim_cust_df,
        (usage_monthly.customer_id == dim_cust_df.customer_id) &
        (usage_monthly.snapshot_time >= dim_cust_df.dw_start_date) &
        (usage_monthly.snapshot_time < dim_cust_df.dw_end_date), how="inner") \
    .join(dim_date_df, usage_monthly.snapshot_date == dim_date_df.full_date, how="left") \
    .withColumn("is_churned", F.when((F.col("is_active") == False) & (F.col("snapshot_date") >=
F.to_date(F.col("dw_start_date"))), True).otherwise(False)) \
    .withColumn("usage_key", F.monotonically_increasing_id()) \
    .select("usage_key", "date_key", "cust_key", "monthly_balance", "clv_ltv", "num_products",
"product_type", "source_system", "is_churned", "month_id")

fact_monthly_usage.write.mode("overwrite") \
    .partitionBy("month_id") \
    .option("path", "/gold/fact_monthly_usage") \
    .saveAsTable("churn_gold.FACT_MONTHLY_USAGE")

# 2. Process Support Tickets Fact
fact_support_tickets = df_ticket_silver \
    .withColumn("ticket_time", F.to_timestamp("submission_date")) \
    .withColumn("ticket_date", F.to_date("submission_date")) \
    .join(dim_cust_df,
        (df_ticket_silver.customer_id == dim_cust_df.customer_id) &

```



```

(F.col("ticket_time") >= dim_cust_df.dw_start_date) &
(F.col("ticket_time") < dim_cust_df.dw_end_date), how="inner") \
.join(dim_date_df, F.col("ticket_date") == dim_date_df.full_date, how="left") \
.withColumn("ticket_key", F.monotonically_increasing_id()) \
.select("ticket_key", "date_key", "cust_key", "ticket_id",
F.col("issue_category").alias("issue_type"),
        "priority_level", "ticket_channel", "assigned_agent", "satisfaction_score",
F.col("resolution_time_hours").alias("resolution_hrs"))

fact_support_tickets.write.mode("overwrite") \
.option("path", "/gold/fact_support_tickets") \
.saveAsTable("churn_gold.FACT_SUPPORT_TICKETS")

```

3. Pipeline Verification

Executes automated SQL queries at runtime to validate pipeline integrity, visualizing SCD2 interval overlaps, partition loads, and surrogate key mappings.

```

#verification
spark.sql("""
    SELECT customer_id, cust_key, age, geography, is_active, dw_start_date, dw_end_date, is_current
    FROM churn_gold.DIM_CUSTOMER
    ORDER BY customer_id, dw_start_date
""").show(20, truncate=False)

spark.sql("""
    SELECT f.month_id, COUNT(*), SUM(CAST(f.is_churned AS INT)) as churned_count
    FROM churn_gold.FACT_MONTHLY_USAGE f
    GROUP BY f.month_id
""").show()

spark.sql("""
    SELECT *
    FROM churn_gold.FACT_MONTHLY_USAGE
""").show()

```



Quit Logout

Files Running Clusters

Select items to perform actions on them.

Upload New ↻

☐ 0 ▾	/ project1 / Gold_layer	Name ▾	Last Modified	File size
	..		seconds ago	
☐	Gold layer.ipynb		Running an hour ago	16.2 kB

```
: #verification
spark.sql("""
    SELECT customer_id, cust_key, age, geography, is_active, dw_start_date, dw_end_date, is_current
    FROM churn_gold.DIM CUSTOMER
    ORDER BY customer_id, dw_start_date
""").show(20, truncate=False)

spark.sql("""
    SELECT f.month_id, COUNT(*), SUM(CAST(f.is_churned AS INT)) as churned_count
    FROM churn_gold.FACT_MONTHLY_USAGE f
    GROUP BY f.month_id
""").show()
```

customer_id	cust_key	age	geography	is_active	dw_start_date	dw_end_date	is_current
CUST-100000	1	37	Cairo, EG	false	2024-06-18 04:45:11	9999-12-31 23:59:59	true
CUST-100001	2	24	Unknown	true	2024-07-15 20:25:57	9999-12-31 23:59:59	true
CUST-100002	3	72	Unknown	true	2025-02-01 23:33:28	9999-12-31 23:59:59	true
CUST-100003	4	74	Alexandria, EG	true	2025-04-24 06:34:34	9999-12-31 23:59:59	true
CUST-100005	5	24	Toronto, CA	true	2025-08-19 11:19:34	9999-12-31 23:59:59	true
CUST-100006	6	74	Riyadh, SA	true	2024-12-11 02:39:29	9999-12-31 23:59:59	true
CUST-100007	7	26	Riyadh, SA	true	2024-06-10 04:09:00	9999-12-31 23:59:59	true
CUST-100008	8	25	Riyadh, SA	true	2024-07-10 05:18:58	9999-12-31 23:59:59	true
CUST-100009	9	20	Sharqia, EG	true	2025-03-05 17:53:51	9999-12-31 23:59:59	true
CUST-100010	10	71	Berlin, DE	false	2024-09-14 19:33:06	9999-12-31 23:59:59	true
CUST-100011	11	51	Toronto, CA	true	2024-06-18 00:28:10	9999-12-31 23:59:59	true

HUE Analysis & Visualization

Business Questions

- 1. Demographic Churn Analysis**
Which age groups and geographic regions exhibit the highest customer churn rates, and how can we leverage this data to design targeted retention campaigns for these specific segments?
- 2. Support Service Impact on Churn**
Is there a direct correlation between delayed support ticket resolution hours or low satisfaction scores and a customer's decision to churn within the same billing month?
- 3. Customer Lifetime Value (CLV) Risk Assessment**
Who are the highest-value customers (High CLV) that have recently churned, and which core products or source systems were they primarily utilizing?
- 4. Product Stickiness & Retention**
Does higher product adoption (using multiple products) increase customer stickiness and reduce churn rates?
- 5. Support Channel Bottlenecks**
Which customer support channels experience the longest resolution times for customers who eventually churn?

Analysis Subjectives

1. Demographic Churn Analysis
 2. Support Service Impact on Churn
 3. Customer Lifetime Value (CLV) Risk Assessment
 4. Product Stickiness & Retention
 5. Support Channel Bottlenecks
-

Queries & Charts

First Query

```
SELECT
  c.geography,
  CASE
    WHEN c.age < 25 THEN 'Under 25'
    WHEN c.age BETWEEN 25 AND 35 THEN '25-35'
    WHEN c.age BETWEEN 36 AND 50 THEN '36-50'
    ELSE 'Over 50'
  END AS age_group,
  COUNT(DISTINCT f.cust_key) AS churned_customers
FROM churn_gold.FACT_MONTHLY_USAGE f
JOIN churn_gold.DIM_CUSTOMER c ON f.cust_key = c.cust_key
WHERE f.is_churned = TRUE
GROUP BY
  c.geography,
  CASE
    WHEN c.age < 25 THEN 'Under 25'
    WHEN c.age BETWEEN 25 AND 35 THEN '25-35'
    WHEN c.age BETWEEN 36 AND 50 THEN '36-50'
    ELSE 'Over 50'
  END
ORDER BY churned_customers DESC;
```

Hue - Editor

localhost:8888/hue/editor?editor=27

Search data and saved documents...

Query History Saved Queries Results (60)

	c.geography	age_group	churned_customers
1	Mumbai, IN	Over 50	92
2	Toronto, CA	Over 50	86
3	Berlin, DE	Over 50	82
4	n/a	Over 50	81
5	unknown	Over 50	80
6	Alexandria, EG	Over 50	78
7	Unknown	Over 50	74
8	Paris, FR	Over 50	73
9	Dubai, AE	Over 50	73
10	New York, US	Over 50	72
11	Giza, EG	Over 50	70
12	Sharqia, EG	Over 50	70
13	London, UK	Over 50	69
14	Cairo, EG	Over 50	66
15	Riyadh, SA	Over 50	64
16	Giza, EG	36-50	54
17	Berlin, DE	36-50	50
18	New York, US	36-50	49
19	Unknown	36-50	48
20	Cairo, EG	36-50	48
21	unknown	36-50	47
22	Toronto, CA	36-50	46

Explanation

To address our first business question, we joined the `fact_monthly_usage` table with the `dim_customers` table. We filtered the data for `is_churned = true` to focus specifically on the customers who have left.

Using a `CASE WHEN` statement, we categorized the customers into distinct age groups for a more meaningful analysis, and then aggregated the total count for each group.

Finally, we selected a bar chart, as it is the most effective way to compare distinct categories, such as age demographic based on customer volume.

Second Query

```
SELECT
    t.satisfaction_score,
    CASE
        WHEN u.is_churned = TRUE THEN 'Churned'
        ELSE 'Retained'
    END AS customer_status,
    COUNT(DISTINCT u.cust_key) AS customer_count
```

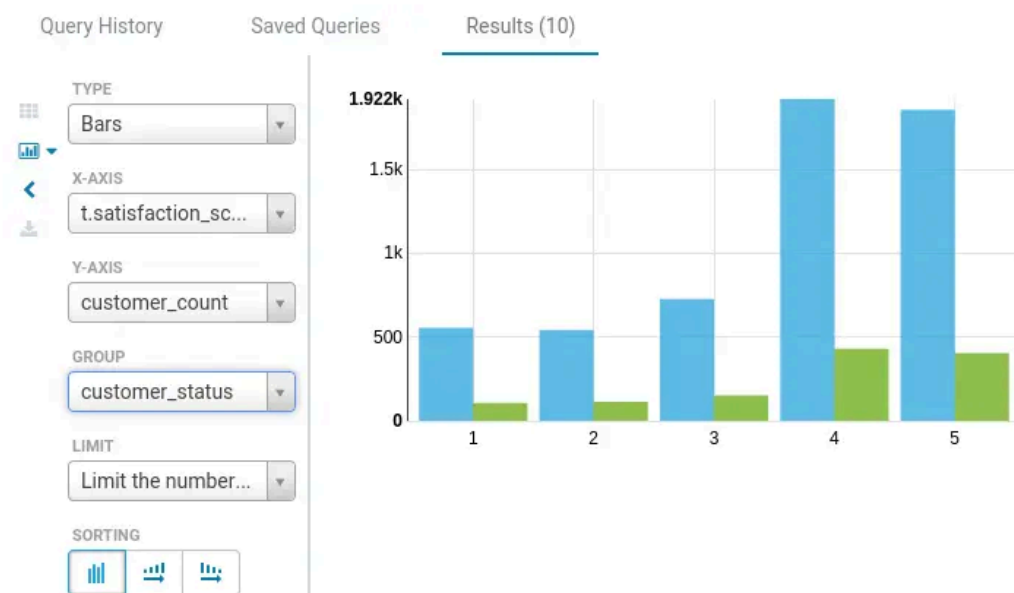
```
FROM churn_gold.FACT_SUPPORT_TICKETS t
JOIN churn_gold.FACT_MONTHLY_USAGE u ON t.cust_key = u.cust_key
GROUP BY
    t.satisfaction_score,
CASE
    WHEN u.is_churned = TRUE THEN 'Churned'
    ELSE 'Retained'
END
ORDER BY
    t.satisfaction_score ASC;
```

Query History

Saved Queries

Results (10)

	t.satisfaction_score	customer_status	customer_count
1	1	Retained	556
2	1	Churned	107
3	2	Retained	543
4	2	Churned	114
5	3	Retained	728
6	3	Churned	152
7	4	Retained	1922
8	4	Churned	431
9	5	Retained	1858
10	5	Churned	405



Explanation

In this second analysis, we joined the tickets table with the usage table using the `cust_key`.

To make the output clearer for business users, we converted the boolean `is_churned` values into descriptive labels ('Churned' and 'Retained').

Finally, the data was grouped by satisfaction score, and a bar chart was used to visualize the relationship between poor service and customer churn.

Third Query

```
SELECT
    product_type,
    COUNT(DISTINCT cust_key) AS churned_customers,
    ROUND(AVG(clv_ltv), 2) AS avg_lost_clv
FROM churn_gold.FACT_MONTHLY_USAGE
WHERE is_churned = TRUE
GROUP BY product_type
ORDER BY avg_lost_clv DESC;
```

	product_type	churned_customers	avg_lost_clv
1	Investment Fund	616	39053.57
2	Checking Account	622	38187.91
3	Credit Card	603	38026.08
4	Mortgage	623	37831
5	Savings Account	597	36968.7
6	Personal Loan	622	36023.09
7	Unknown	100	35360.02



Explanation

In this analysis, we focused specifically on churned customers to identify the most unsatisfactory products driving customer attrition.

Furthermore, we calculated the average revenue loss associated with each of these products. A chart was utilized to visually compare the financial impact across different products, making it easier for stakeholders to quickly pinpoint which product requires immediate attention.

Fourth Query

```
SELECT
    num_products,
    COUNT(DISTINCT cust_key) AS total_customers,
    SUM(CAST(is_churned AS INT)) AS churned_customers,
    ROUND(
        (SUM(CAST(is_churned AS INT)) * 100.0 / COUNT(DISTINCT cust_key)),
        2
    ) AS churn_rate_percentage
FROM churn_gold.FACT_MONTHLY_USAGE
```

```
GROUP BY num_products
ORDER BY num_products;
```

Query History		Saved Queries		Results (5)	
		num_products	total_customers	churned_customers	churn_rate_percentage
	1	1	2970	843	28.38
	2	2	2902	792	27.29
	3	3	2870	837	29.16
	4	4	2900	776	26.76
	5	5	2865	823	28.73



Explanation

In this analysis, we segmented the customer base according to the number of products they are subscribed to (product tiers).

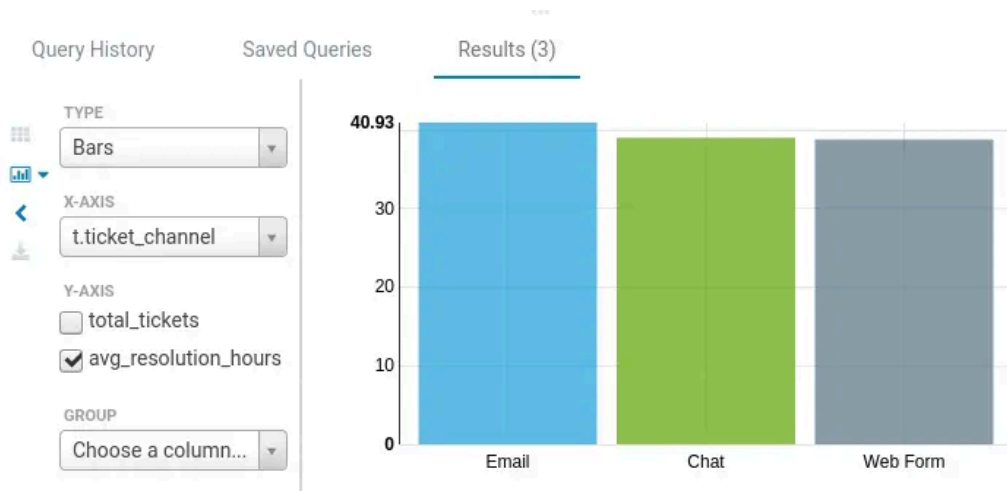
We then calculated the total number of unique customers and the exact number of churned customers within each tier by casting the boolean `is_churned` flag into an integer and summing the values.

After calculating the churn percentage and rounding the results for clarity, we visualized the findings.

The accompanying chart clearly illustrates an inverse relationship: customers with fewer product subscriptions exhibit a significantly higher churn rate, highlighting the importance of product adoption in customer retention.

Fifth Query

```
SELECT
    t.ticket_channel,
    COUNT(t.ticket_key) AS total_tickets,
    ROUND(AVG(t.resolution_hrs), 2) AS avg_resolution_hours
FROM churn_gold.FACT_SUPPORT_TICKETS t
JOIN churn_gold.FACT_MONTHLY_USAGE u ON t.cust_key = u.cust_key
WHERE u.is_churned = TRUE
GROUP BY t.ticket_channel
ORDER BY avg_resolution_hours DESC;
```



Explanation

We joined `FACT_SUPPORT_TICKETS` with `FACT_MONTHLY_USAGE`, filtering exclusively for churned customers to analyze the drivers of their departure.

By grouping the data by communication channel, we calculated total ticket volumes and average resolution times.

The accompanying chart highlights operational bottlenecks by showing which channels suffered the longest delays.

This insight enables the business to reallocate resources effectively, optimize response times, and improve future retention.